Google Summer of Code 2026
(The OpenMRS Project)

Notification : Intelligent Lab Result & Prescription Alerts for OpenMRS

Personal Info

Name : Mayank Kumar
Discord Handle : _vortex_m (1191252308282060830)
Portfolio : yourmayankdev.xyz
E-mail : mail.mayank001@gmail.com
GitHub handle : vortex-m
Nationality : India
Primary Language Spoken : English
Time Zone: India (UTC + 5:30)

University Info

University: Chandigarh University, Mohali, Punjab, India
Major: Computer Science and Engineering
Current Year: 3rd year (May 2027 expected graduation)
Degree: Bachelor of Technology (B.Tech)

1. Abstract

Clinicians in resource-constrained healthcare settings spend significant time manually tracking whether lab results are ready — time that should be spent on patient care. This project implements a **Smart Notification System** in OpenMRS that uses an intelligent Clinical Priority Filter to push real-time alerts *only* when a result warrants immediate action, while silently updating the chart for routine findings. The result: faster clinical decisions, reduced alert fatigue, and better patient outcomes.

2. Problem Statement

2.1 The Manual Tracking Burden

When a clinician orders a laboratory test in OpenMRS, there is currently no automated mechanism to alert them when results are ready. This creates a cascade of inefficiencies:

- Must **periodically poll** the system for new results, interrupting ongoing consultations
- Cannot distinguish **urgent results from routine ones** without manually reviewing each
- Risks **missing follow-ups** during high-volume clinic sessions
- Delays treatment when results are time-sensitive — a critical patient safety risk

2.2 The Alert Fatigue Paradox

A naive solution — notify for every result — creates a different and equally dangerous problem. Research in clinical informatics consistently shows that when clinicians receive too many notifications, they become desensitized and begin ignoring them. Critical alerts get buried in noise. The right solution is not more notifications — it is **smarter notifications**

3. Proposed Solution

3.1 Clinical Priority Filter

The core of this system is a filtering layer that evaluates every incoming lab result against two criteria before deciding whether to notify:

Result Type	Action	Rationale
STAT / Urgent Order	**Immediate Push Notification**	**Clinician is actively waiting to make treatment decisions**
Critical / Dangerous Value	**Immediate Push Notification**	**Life-threatening findings require immediate intervention**
Routine Normal Result	**Silent Chart Update**	**No interruption needed; review during scheduled rounds**
Slightly Abnormal Result	**Silent Chart Update**	**Non-critical; does not change immediate management**
Expected Abnormal	**Silent Chart Update**	**e.g., elevated creatinine in known kidney disease - expected**

3.2 Use Cases

Use Case 1 — Lab Result Notification

1. Clinician orders a lab test, setting priority (STAT vs. Routine)
2. Lab technician processes sample and enters final results into OpenMRS
3. OpenMRS fires a **lab result created** event
4. **NotificationTriggerService** intercepts the event and passes it to the **ClinicalPriorityFilterService**
5. Filter evaluates: STAT flag OR critical value threshold exceeded?
6. **If YES** ® Persist notification + push to clinician: *"CRITICAL: John Doe — Potassium 2.2"*
7. **If NO** ® Silently update patient chart; no notification generated

Use Case 2 — Prescription Safety Alert

1. Patient presents prescription at pharmacy
2. Pharmacist identifies drug interaction / dosage concern and flags it in the system
3. System evaluates flag: Drug Interaction, Allergy Conflict, or Dosage Concern?
4. **If YES** ® Immediate push to prescribing clinician: *SAFETY ALERT: Jane Doe — Metformin interaction"*
5. Clinician approves modification or sends alternative; pharmacy is notified of resolution

3.3 System Architecture

The system is structured as a new OpenMRS module with four layers:

Layer	Component	Responsibility
Event Layer	OpenMRS Event Module hook	Listens for EncounterSaveEvent / OrderSaveEvent
Filter Layer	ClinicalPriorityFilterService	Evaluates STAT flag + critical value thresholds
Persistence	NotificationDAO + notification table	Stores notifications with acuity, read status, timestamp
API Layer	NotificationRestController	REST endpoints: list, count, mark-read (paginated)
Frontend	React Notification Center (OWA)	Bell icon, inbox UI, actionable alert cards

4. Technical Implementation

4.1 Backend — Event Listener

OpenMRS's Event Module publishes events when domain objects are saved. We register a listener on the **obs-saved** topic:

EventListener

```
@EventSubscriber
public void onObsSaved(ObsSavedEvent event) {
    Obs obs = event.getObs();
    Order order = obs.getOrder();

    if (order != null) {
        NotificationContext ctx = new NotificationContext(obs, order);
        notificationTriggerService.evaluate(ctx);
    }
}
```

4.2 Backend — Priority Filter Service

The filter applies two checks in order — STAT flag first, then critical value threshold:

ClinicalPriorityFilterService

```java
public AcuityLevel evaluate(NotificationContext ctx) {

    // Check 1: Was the order marked STAT / URGENT?
    if (Order.Urgency.STAT.equals(ctx.getOrder().getUrgency())) {
        return AcuityLevel.STAT;
    }

    // Check 2: Does the value breach a critical threshold?
    CriticalThreshold threshold = thresholdService
        .getThreshold(ctx.getObs().getConcept());

    if (threshold != null && threshold.isCritical(ctx.getObs().getValueNumeric())) {
        return AcuityLevel.CRITICAL;
    }

    return AcuityLevel.ROUTINE; // No notification generated
}
```

4.3 Database Schema

A single new table stores all notifications:

Notification Table

```sql
CREATE TABLE notification (

    notification_id INT NOT NULL AUTO_INCREMENT,

    recipient_id INT NOT NULL, -- FK: users.user_id
    patient_id INT NOT NULL, -- FK: patient.patient_id

    acuity_level VARCHAR(20) NOT NULL, -- STAT | CRITICAL | ROUTINE
    message VARCHAR(500) NOT NULL,

    trigger_type VARCHAR(50) NOT NULL, -- LAB_RESULT | RX_SAFETY

    is_read TINYINT(1) DEFAULT 0,

    created_at DATETIME NOT NULL,
    read_at DATETIME,

    PRIMARY KEY (notification_id),
    INDEX idx_recipient_read (recipient_id, is_read)
);
```

4.4 REST API Design

Method	Endpoint	Description
GET	/ws/rest/v1/notification	Fetch notifications (paginated, filterable by acuity)
GET	/ws/rest/v1/notification/count	Unread count for bell icon badge
POST	/ws/rest/v1/notification/{id}/read	Mark single notification as read
POST	/ws/rest/v1/notification/readAll	Mark all notifications as read

5.5 Frontend - Notification Center

A React component integrated into the OpenMRS SPA provides:

- **Bell icon** in the top navigation bar with an animated red badge showing unread count
- **Notification inbox** with tabs: All/Unread/Critical each showing patient name, message, and timestamp
- **Critical alert card** rendered with a red left border and urgent icon; clicking navigates directly to the patient chart
- **Mark as read / Mark all as read** controls with optimistic UI updates
- **Empty state** when no unread notifications exist — no unnecessary clutter

5. Testing Strategy

Given the patient-safety criticality of this feature, testing is a first-class deliverable — not an afterthought. The strategy has three tiers:

5.1 Unit Tests (JUnit 5 + Mockito)

Unit Test

```java
@Test
void statOrderShouldTriggerNotification() {

    Order mockOrder = buildOrder(Order.Urgency.STAT);
    Obs mockObs = buildObs(mockOrder, 5.0);

    AcuityLevel level = filterService.evaluate(
        new NotificationContext(mockObs, mockOrder)
    );

    assertEquals(AcuityLevel.STAT, level);
}

@Test
void routineNormalResultShouldNotTrigger() {

    Order mockOrder = buildOrder(Order.Urgency.ROUTINE);
    Obs mockObs = buildObs(mockOrder, 4.2); // Within normal range

    AcuityLevel level = filterService.evaluate(
        new NotificationContext(mockObs, mockOrder)
    );

    assertEquals(AcuityLevel.ROUTINE, level);
}
```

5.2 Integration Tests (REST API)

Using OpenMRS's built-in test infrastructure (**BaseModuleWebContextSensitiveTest**), each REST endpoint will be tested end-to-end against an in-memory H2 database, verifying correct HTTP status codes, pagination parameters, and JSON response structure.

5.3 Edge Case Coverage

- Lab result with **missing concept mapping** — no threshold configured; should default to ROUTINE
- Multiple concurrent critical results for the same patient — deduplication within a 5-minute window
- **Pharmacist flags prescription** for a clinician who is not currently logged in — notification persists for later retrieval
- Marking notification as read is **idempotent** — double-read does not error
- API response when recipient has **zero notifications** — returns empty array, not 404

6. Deliverables

#	Deliverable	Description
1	NotificationTriggerModule	Event listener + priority filter service + DAO + DB migration.
2	REST API	4 endpoints with pagination, filtering, mark-read functionality.
3	Admin Config UI	Allow admins to set critical value thresholds per OpenMRS Concept.
4	React Notification Center	Bell icon, inbox, critical card, mark-read — integrated into OWA.
5	Test Suite	Unit tests for filter logic, integration tests for REST, UI component tests
6	Documentation	Developer guide: how to add a new notification trigger type.

7. Timeline & Milestones

Period	Dates	Milestone / Tasks
Pre-GSoC	Now – May 8	Set up OpenMRS dev environment · Study Event Module + REST module documents· Post intro on OpenMRS Talk
Community Bonding	May 8 – May 26	Finalise notification DB schema with mentors· Agree on critical value threshold config approach · Review existing codebase notification pattern.
Phase 1 Week 1–2	May 27 – Jun 8	Implement NotificationTriggerService (event hook) · Write unit tests for trigger service
Phase 1 Week 3–4	Jun 9 – Jun 22	Implement ClinicalPriorityFilterService · DB schema + Liquibase migration + NotificationDAO
Phase 1 Week 5–6	Jun 23 – Jul 6	Build all 4 REST API endpoints· Integration tests for REST layer
Mid-Term Eval	Jul 7 – Jul 13	Working backend: events → filter → persist → REST API functional
Phase 2 Week 7–8	Jul 14 – Jul 27	Frontend: bell icon + notification inbox UI· Connect frontend to REST API
Phase 2 Week 9	Jul 28 – Aug 3	Prescription safety alert flow (Use Case 2)· Pharmacist → clinician notification + tracking
Phase 3 Week 10–11	Aug 4 – Aug 17	Admin config UI for critical thresholds· End-to-end testing · Performance review · Bug fixes
Phase 3 Week 12–13	Aug 18 – Aug 25	Developer documentation· Code cleanup· Final PR submitted
Final Evaluation	Aug 25	All 6 deliverables complete, documented, and merged

8. Why I Am the Right Contributor

8.1 Relevant Technical Experience

- **Intermediate Java proficiency** - built multiple Java projects including Microfinance Loan Management Service.
- **Event-driven architecture understanding** - familiar with publish/subscribe patterns from AlgoHub.
- **REST API design** - designed and consumed REST APIs in Microfinance Loan Management Serv.
- **Database design** - comfortable writing SQL schemas, indexes, and migrations
- **React / frontend basics** - built UI components in Algohub

9.2 Open Source Readiness

While this will be my first open-source contribution, I have prepared deliberately: I have set up the OpenMRS SDK development environment locally, read through the Event Module and REST Module documentation, and studied how existing notification patterns work in the codebase. I have already posted an introduction on the OpenMRS Talk forum and initiated a conversation with the project mentors to get early feedback on this architecture before submission.

10. Future Goals (Post-GSoC)

- **WebSocket push** — replace REST polling with real-time WebSocket delivery for instant notifications without page refresh.
- **Mobile push notifications** — extend to OpenMRS Android Client so clinicians receive alerts on their Phones.
- **Notification analytics dashboard** — aggregate data on which alert types fire most frequently, response times, and outcomes to enable quality improvement.
- **Pluggable trigger framework** — make the notification system extensible so other OpenMRS modules can register new trigger types without modifying core notification code.
- **Expand to appointment reminders** — notify clinicians of upcoming patient appointments that need Preparation.

Thank you for considering this proposal. I am genuinely excited about the opportunity to contribute to OpenMRS and to build a feature that will improve patient safety in clinics around the world. I look forward to collaborating with @Kennedy Makombe, @Donald Kibet, and the broader OpenMRS community.